

SQL solutions

1. User Upcoming Events

Show a list of all upcoming events a user is registered for in their city, sorted by date.

```
SELECT u.user_id, u.full_name, e.title, e.city, e.start_date
FROM Users u
JOIN Registrations r ON u.user_id = r.user_id
JOIN Events e ON r.event_id = e.event_id
WHERE e.status = 'upcoming'
AND e.city = u.city
ORDER BY e.start_date ASC;
```

2. Top Rated Events

Identify events with the highest average rating, considering only those that have received at least 10 feedback submissions.

```
SELECT e.event_id, e.title, ROUND(AVG(f.rating), 2) AS avg_rating,
COUNT(f.feedback_id) AS feedback_count
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(f.feedback_id) >= 10
ORDER BY avg_rating DESC;
```

3. Inactive Users

Retrieve users who have not registered for any events in the last 90 days.

```
SELECT u.user_id, u.full_name, u.email
FROM Users u
WHERE u.user_id NOT IN (
    SELECT r.user_id
    FROM Registrations r
    WHERE r.registration_date >= CURDATE() - INTERVAL 90 DAY
);
```

4. Peak Session Hours

Count how many sessions are scheduled between 10 AM to 12 PM for each event

```
SELECT e.event_id, e.title,
COUNT(s.session_id) AS sessions_in_peak_hours
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
AND TIME(s.start_time) >= '10:00:00'
AND TIME(s.start_time) < '12:00:00'
```

SQL solutions

```
GROUP BY e.event_id, e.title  
ORDER BY sessions_in_peak_hours DESC;
```

5. Most Active Cities

List the top 5 cities with the highest number of distinct user registrations.

```
SELECT u.city, COUNT(DISTINCT r.user_id) AS distinct_registrations  
FROM Users u  
JOIN Registrations r ON u.user_id = r.user_id  
GROUP BY u.city  
ORDER BY distinct_registrations DESC  
LIMIT 5;
```

6. Event Resource Summary

Generate a report showing the number of resources (PDFs, images, links) uploaded for each event.

```
SELECT e.event_id, e.title,  
       COUNT(r.resource_id) AS total_resources,  
       SUM(r.resource_type = 'pdf') AS pdf_count,  
       SUM(r.resource_type = 'image') AS image_count,  
       SUM(r.resource_type = 'link') AS link_count  
FROM Events e  
LEFT JOIN Resources r ON e.event_id = r.event_id  
GROUP BY e.event_id, e.title;
```

7. Low Feedback Alerts

List all users who gave feedback with a rating less than 3, along with their comments and associated event names.

```
SELECT u.user_id, u.full_name, e.title AS event_name,  
       f.rating, f.comments, f.feedback_date  
FROM Feedback f  
JOIN Users u ON f.user_id = u.user_id  
JOIN Events e ON f.event_id = e.event_id  
WHERE f.rating < 3  
ORDER BY f.rating ASC;
```

8. Sessions per Upcoming Event

Display all upcoming events with the count of sessions scheduled for them.

SQL solutions

```
SELECT e.event_id, e.title, e.city, e.start_date,  
       COUNT(s.session_id) AS session_count  
FROM Events e  
LEFT JOIN Sessions s ON e.event_id = s.event_id  
WHERE e.status = 'upcoming'  
GROUP BY e.event_id, e.title, e.city, e.start_date  
ORDER BY e.start_date;
```

9. Organizer Event Summary

For each event organizer, show the number of events created and their current status (upcoming, completed, cancelled).

```
SELECT u.user_id, u.full_name,  
       COUNT(e.event_id) AS total_events,  
       SUM(e.status = 'upcoming') AS upcoming,  
       SUM(e.status = 'completed') AS completed,  
       SUM(e.status = 'cancelled') AS cancelled  
FROM Users u  
JOIN Events e ON u.user_id = e.organizer_id  
GROUP BY u.user_id, u.full_name  
ORDER BY total_events DESC;
```

10. Feedback Gap

Identify events that had registrations but received no feedback at all.

```
SELECT e.event_id, e.title  
FROM Events e  
WHERE e.event_id IN (SELECT DISTINCT event_id FROM Registrations)  
AND e.event_id NOT IN (SELECT DISTINCT event_id FROM Feedback);
```

11. Daily New User

Count Find the number of users who registered each day in the last 7 days.

```
SELECT registration_date, COUNT(user_id) AS new_users  
FROM Users  
WHERE registration_date >= CURDATE() - INTERVAL 7 DAY  
GROUP BY registration_date  
ORDER BY registration_date DESC;
```

12.Event with Maximum Sessions

SQL solutions

List the event(s) with the highest number of sessions.

```
SELECT e.event_id, e.title, COUNT(s.session_id) AS session_count
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(s.session_id) = (
    SELECT MAX(cnt) FROM (
        SELECT COUNT(session_id) AS cnt
        FROM Sessions
        GROUP BY event_id
    ) AS sub
);
```

13. Average Rating per City

Calculate the average feedback rating of events conducted in each city.

```
SELECT e.city, ROUND(AVG(f.rating), 2) AS avg_rating,
    COUNT(f.feedback_id) AS total_feedback
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.city
ORDER BY avg_rating DESC;
```

14. Most Registered Events

List top 3 events based on the total number of user registrations.

```
SELECT e.event_id, e.title, COUNT(r.registration_id) AS total_registrations
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY total_registrations DESC
LIMIT 3;
```

15. Event Session Time Conflict

Identify overlapping sessions within the same event (i.e., session start and end times that conflict).

```
SELECT s1.event_id, s1.session_id AS session_1, s1.title AS title_1,
    s2.session_id AS session_2, s2.title AS title_2,
```

SQL solutions

```
s1.start_time AS start_1, s1.end_time AS end_1,  
s2.start_time AS start_2, s2.end_time AS end_2  
FROM Sessions s1  
JOIN Sessions s2  
ON s1.event_id = s2.event_id  
AND s1.session_id < s2.session_id  
AND s1.start_time < s2.end_time  
AND s1.end_time > s2.start_time;
```

16. Unregistered Active Users

Find users who created an account in the last 30 days but haven't registered for any events.

```
SELECT u.user_id, u.full_name, u.email, u.registration_date  
FROM Users u  
WHERE u.registration_date >= CURDATE() - INTERVAL 30 DAY  
AND u.user_id NOT IN (  
    SELECT DISTINCT user_id FROM Registrations  
);
```

17. Multi-Session Speakers

Identify speakers who are handling more than one session across all events.

```
SELECT speaker_name, COUNT(session_id) AS total_sessions,  
    GROUP_CONCAT(title ORDER BY start_time SEPARATOR ' | ') AS sessions  
FROM Sessions  
GROUP BY speaker_name  
HAVING COUNT(session_id) > 1  
ORDER BY total_sessions DESC;
```

18. Resource Availability Check

List all events that do not have any resources uploaded.

```
SELECT e.event_id, e.title, e.status  
FROM Events e  
WHERE e.event_id NOT IN (  
    SELECT DISTINCT event_id FROM Resources  
);
```

SQL solutions

19. Completed Events with Feedback Summary

For completed events, show total registrations and average feedback rating.

```
SELECT e.event_id, e.title,  
       COUNT(DISTINCT r.registration_id) AS total_registrations,  
       ROUND(AVG(f.rating), 2) AS avg_rating,  
       COUNT(DISTINCT f.feedback_id) AS feedback_count  
FROM Events e  
LEFT JOIN Registrations r ON e.event_id = r.event_id  
LEFT JOIN Feedback f ON e.event_id = f.event_id  
WHERE e.status = 'completed'  
GROUP BY e.event_id, e.title;
```

20. User Engagement Index

For each user, calculate how many events they attended and how many feedbacks they submitted.

```
SELECT u.user_id, u.full_name,  
       COUNT(DISTINCT r.event_id) AS events_registered,  
       COUNT(DISTINCT f.feedback_id) AS feedbacks_submitted  
FROM Users u  
LEFT JOIN Registrations r ON u.user_id = r.user_id  
LEFT JOIN Feedback f ON u.user_id = f.user_id  
GROUP BY u.user_id, u.full_name  
ORDER BY events_registered DESC, feedbacks_submitted DESC;
```

21. Top Feedback Providers

List top 5 users who have submitted the most feedback entries.

```
SELECT u.user_id, u.full_name, COUNT(f.feedback_id) AS feedback_count  
FROM Users u  
JOIN Feedback f ON u.user_id = f.user_id  
GROUP BY u.user_id, u.full_name  
ORDER BY feedback_count DESC  
LIMIT 5;
```

22. Duplicate Registrations Check

SQL solutions

Detect if a user has been registered more than once for the same event.

```
SELECT user_id, event_id, COUNT(*) AS registration_count
FROM Registrations
GROUP BY user_id, event_id
HAVING COUNT(*) > 1;
```

23. Registration Trends

Show a month-wise registration count trend over the past 12 months.

```
SELECT DATE_FORMAT(registration_date, '%Y-%m') AS month,
       COUNT(*) AS total_registrations
FROM Registrations
WHERE registration_date >= CURDATE() - INTERVAL 12 MONTH
GROUP BY DATE_FORMAT(registration_date, '%Y-%m')
ORDER BY month ASC;
```

24. Average Session Duration per Event

Compute the average duration (in minutes) of sessions in each event.

```
SELECT e.event_id, e.title,
       ROUND(AVG(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)), 1) AS
avg_duration_mins
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id
GROUP BY e.event_id, e.title
ORDER BY avg_duration_mins DESC;
```

25. Events Without Sessions

List all events that currently have no sessions scheduled under them.

```
SELECT e.event_id, e.title, e.status, e.start_date FROM Events e WHERE
e.event_id NOT IN ( SELECT DISTINCT event_id FROM Sessions ) ORDER BY
e.start_date;
```